

The Fluid Toy

Victor Do | Cameron Tuffner-Lyons | Davis Cheung | JJ Marr
Optimizing Collision Detection with GPU Acceleration for Real-Time Particle-Based Fluid Simulations.

Play Here



TheFluidToy.com

MOTIVATION

With the explosion of GPU computing power in recent years driven by AI innovations, **parallel computing** has reached unprecedented heights. Inspired by this, we set out to target a highly parallelized application to demonstrate the possibilities of leveraging this power. Recognizing that many aspiring game developers begin their journeys with **The Unity Game Engine** for its accessibility, we identified a significant gap: the **lack of a built-in fluid simulation** system, which makes implementing dynamic fluids both challenging and costly. To address this, we developed **The Fluid Toy**; a powerful yet accessible **realtime fluid simulation engine** designed to harness the full capabilities of both **CPU** and **GPU** processors. By removing financial and technical barriers, The Fluid Toy empowers indie developers to push performance limits and craft richer, more immersive gaming experiences.

APPROACH

The Fluid Toy implements **Smoothed Particle Hydrodynamics (SPH)** fluid simulation, a **fully particle-based** method ideal for modeling complex, dynamic free-surface flows such as splashes and breaking waves. Unlike grid-based methods, SPH's grid-less nature simplifies handling **irregular geometry** boundaries and dynamic topologies, with the capability for adaptive resolution where detail is most needed. The Fluid Toy also supports the simulation of **multiple types of fluids and solids**, showcasing SPH's versatility in modeling viscous fluids, **multiphase flows**, and **deformable solids**.

To optimize computational performance, collision detection and physics calculations are distributed across **CPU and GPU** portions, leveraging **heterogeneous computation**. This design enhances scalability, enabling efficient simulations even as the complexity of fluid interactions increases. Additionally, SPH's **memory efficiency**—stemming from its grid-less approach—contributes to its suitability for **realtime** dynamically changing boundary conditions. While hybrid methods like FLIP excel in high-resolution scenarios, SPH offers distinct advantages in adaptability and ease of implementation, making it a fitting choice for this project.

DESIGN & IMPLEMENTATION

The Fluid Toy employs a **five-stage, parallelized SPH simulation** framework to handle collision detection and particle physics calculations. Each particle is represented in an **Array of Structs** with data for **position, velocity, pressure, type** and **temperature**. Spatial subdivision and **counting sort** techniques are utilized to optimize neighbor search efficiency for collision detection. To achieve scalable performance, computational workloads are distributed between the CPU and GPU, leveraging heterogeneous computation. Physics calculations are derived from SPH-based Navier-Stokes equations, **enabling real-time simulation of dense, variable-sized particle systems at scale**.

SPH FLUID SIMULATION ALGORITHM

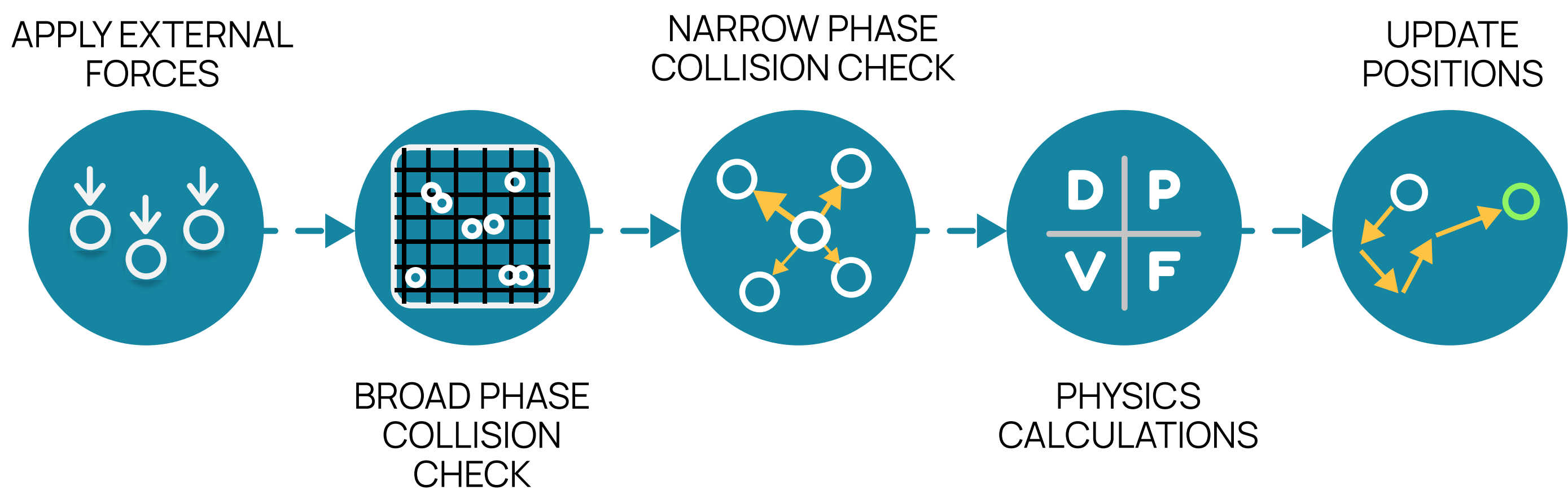


Diagram 1. Order of operations for SPH Algorithm

When using a **uniform spatial grid** for the **broad-phase** particle collision detection, each particle only needs to check adjacent grid squares for potential collisions. A limitation arises from this however as **grid regions** must **match the size of the largest particle** (otherwise collisions might go undetected). This can lead to inefficiencies during broad-phase collision detection when smaller particles are confined to over sized grid cells, creating computational bottlenecks in simulations with particles of varying sizes.

To resolve this, we decoupled the simulation: **small particles** are processed by the **GPU** (using dense, uniform spatial grids) while **large obstacles** are managed by the **CPU** (with sweep and prune algorithms). This hybrid approach eliminates wasted computation and maintains optimal conditions for both scales.

FLUID SIMULATION RUNNING IN REAL-TIME

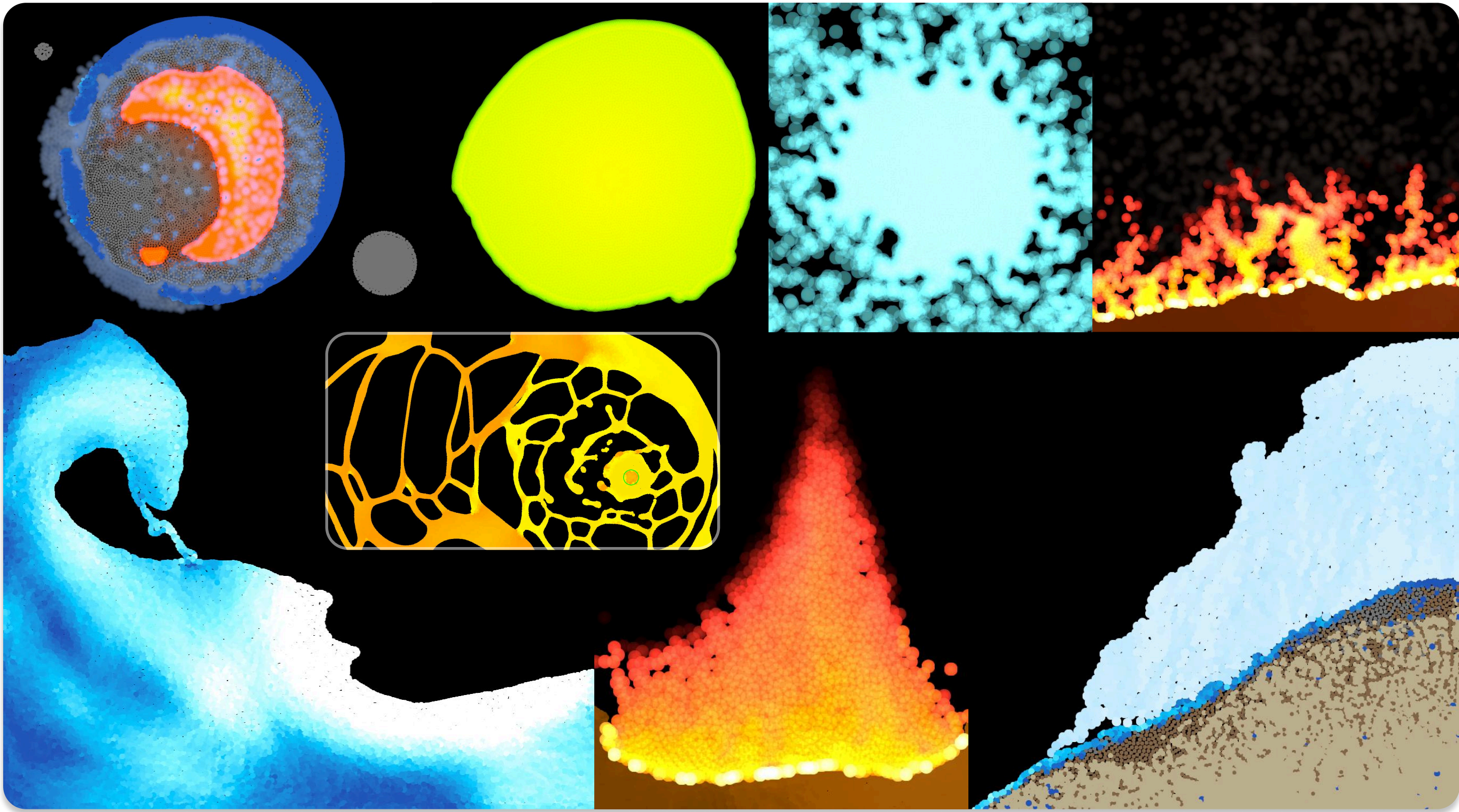
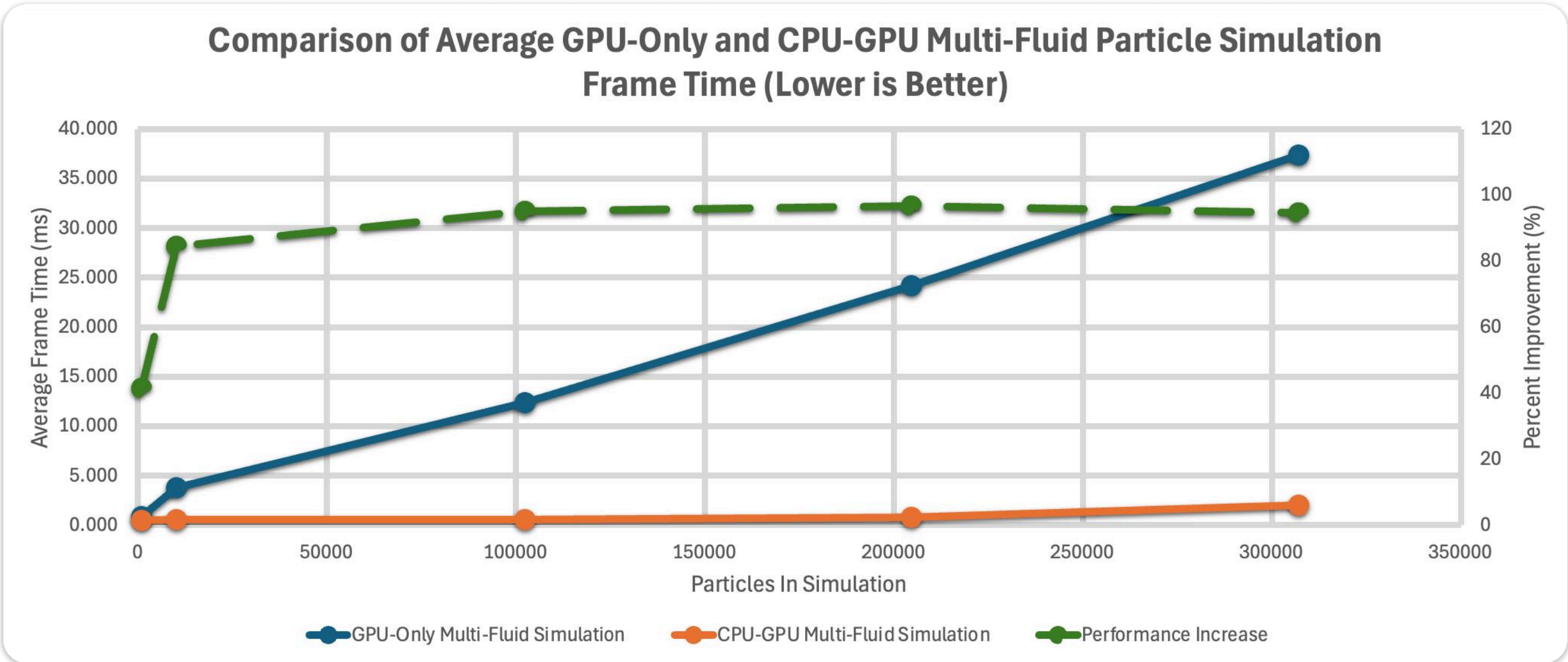


Table 1. (Left to Right, Top to bottom) Images of Planets, Radioactive waste, Neutrons, Incomplete Combustion, Honey, Waves, Oil on fire, Glacier melting

RESULTS

In testing, our **heterogeneous implementation** of the **SPH** fluid simulation was able to achieve up to a **82% reduction** in frame time. This increase in performance was achieved through the heterogeneous split as well as various optimization techniques derived through experimentation including GPU kernel consolidation, particle position randomization on deactivation, particle de-clumping, choice of data structures, GPU memory alignment and choice of sorting algorithm. The **performance increase** of the **heterogeneous implementation scaled** with the **complexity** of the simulation, improving scalability of the simulation for **higher particle counts**.



Graph 1. Performance Comparison of GPU-Only vs CPU-GPU Multi-Fluid Simulations of Two Fluids With Significantly Different Particle Sizes.

CONCLUSION

The Fluid Toy demonstrates how SPH can be optimized for **real-time applications** through **heterogeneous CPU/GPU computing**. By implementing a hybrid device approach to spatial subdivision, we efficiently resolved the performance limitations of uniform grids in **multi-scale particle simulations**. Our hybrid architecture—processing small particles on the GPU and large obstacles on the CPU—achieved up to a **82% reduction** in frame time, enabling **scalable, high-density fluid interactions** while preserving physical accuracy.

The performance gains from this approach enabled more advanced computational features in our simulation, including **temperature** and **friction** simulation. Usage of **SPH** allowed for dynamic boundary conditions, multi-phase fluid interactions, and complex fluid-obstacle collision detection. The Fluid Toy's architecture thus provides a foundation for **high-performance particle-based simulations**, proving that complex fluid dynamics can be achieved efficiently and practically in heterogeneous computing environments.

REFERENCES & ACKNOWLEDGEMENTS

- Dr. Gul N. Khan, P.Eng.
- D. Koschier, J. Bender, B. Solenthaler, and M. Teschner. (2019) Smoothed particle hydrodynamics techniques for the physics based simulation of fluids and solids. [Online]. Available: [https://sph-tutorial.physics-simulation.org/pdf/SPH Tutorial.pdf](https://sph-tutorial.physics-simulation.org/pdf/SPH%20Tutorial.pdf)